

PROYECTO: MISIÓN ÓRBITA

ENTREGABLE AVANCE 1 — PREPARACIÓN DE LA MISIÓN

Curso:	Principios de Programación 1	Código:	SOFT-01
Sección:	SCV3	Periodo:	C3-2026
Docente:	Verónica Mora Lezcano	Fecha:	14 de Octubre de 2026

INFORMACIÓN DEL GRUPO & REPOSITORIO GIT

Integrantes del Equipo:

- Estudiante 1: [Nombre Apellido 1] — Cédula / Carné: [00000000]
- Estudiante 2: [Nombre Apellido 2] — Cédula / Carné: [00000000]
- Estudiante 3: [Nombre Apellido 3] — Cédula / Carné: [00000000]

Enlace al Repositorio de GitHub:

<https://github.com/usuario-grupo/mision-orbita>

RESUMEN DEL AVANCE 1

El presente documento contiene la solución completa para el Avance 1 del proyecto integrador **Misión Órbita**. En esta fase inicial, desarrollamos cuatro algoritmos independientes orientados a la preparación de la exploración espacial, aplicando conceptos del paradigma imperativo en Python como entrada/salida por consola, variables simples, constantes declaradas explícitamente, operaciones aritméticas básicas y estructuras condicionales simples y dobles (sin anidamiento ni ciclos).

Para garantizar el cumplimiento riguroso de los estándares de la Universidad CENFOTEC y la rúbrica de evaluación, cada ejercicio incluye su respectiva tabla de variables estandarizada, diagrama de flujo estructurado, código fuente comentado bajo el estándar de codificación exigido y la matriz de casos de prueba con sus evidencias ejecutadas en consola.

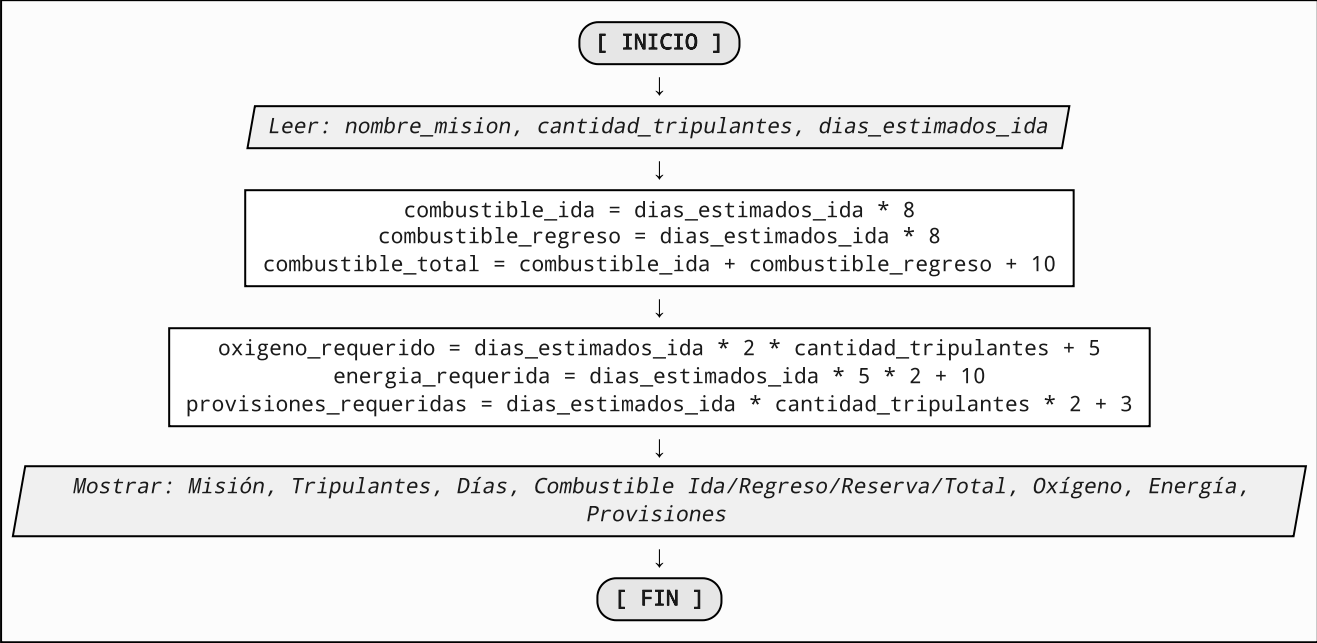
EJERCICIO 1: CÁLCULO DE RECURSOS NECESARIOS PARA UNA MISIÓN

Este algoritmo determina la cantidad exacta de combustible (ida, regreso y reserva), oxígeno, energía y provisiones necesarias para planificar un viaje espacial a partir de la duración del trayecto y la cantidad de tripulantes.

1.1 TABLA DE VARIABLES

NOMBRE	TIPO	CLASIFICACIÓN	DESCRIPCIÓN / VALOR INICIAL
RESERVA_COMBUSTIBLE	int	Constante	Reserva fija de combustible para emergencias (10 unidades).
EMERGENCIA_OXIGENO	int	Constante	Oxígeno extra de seguridad para la misión (5 unidades).
EXPLORACION_ENERGIA	int	Constante	Energía fija requerida para labores de exploración (10 unidades).
EXTRA_PROVISIONES	int	Constante	Provisiones adicionales de emergencia (3 unidades).
nombre_mision	str	Entrada	Nombre identificador de la misión espacial.
cantidad_tripulantes	int	Entrada	Número total de astronautas en la nave.
dias_estimados_ida	int	Entrada	Días proyectados para llegar al destino.
combustible_ida / regreso	int	Intermedia	Combustible calculado para cada tramo (días × 8).
combustible_total / oxigeno_total...	int	Salida	Totales calculados según fórmulas de la agencia.

1.2 DIAGRAMA DE FLUJO



1.3 CÓDIGO FUENTE EN PYTHON (EJERCICIO1_RECURSOS.PY)

```
# =====
# Archivo: ejercicio1_recursos.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV3
# Integrantes: [Nombre 1], [Nombre 2], [Nombre 3]
# Fecha: 14/10/2026 | Versión: 1.0
# Descripción: Calcula los recursos totales requeridos para una misión espacial.
# =====

# 1. Declaración de Constantes
CONSUMO_COMBUSTIBLE_DIA = 8
RESERVA_COMBUSTIBLE = 10
CONSUMO_OXIGENO_TRIPULANTE_DIA = 2
EMERGENCIA_OXIGENO = 5
CONSUMO_ENERGIA_DIA = 5
EXPLORACION_ENERGIA = 10
CONSUMO_PROVISIONES_TRIPULANTE_DIA = 1
EXTRA_PROVISIONES = 3

# 2. Entrada de Datos
nombre_mision = input("Ingrese el nombre de la misión: ")
cantidad_tripulantes = int(input("Ingrese la cantidad de tripulantes: "))
dias_estimados_ida = int(input("Ingrese los días estimados hasta el destino: "))

# 3. Procesamiento y Cálculos
combustible_ida = dias_estimados_ida * CONSUMO_COMBUSTIBLE_DIA
combustible_regreso = dias_estimados_ida * CONSUMO_COMBUSTIBLE_DIA
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE

oxigeno_requerido = (dias_estimados_ida * CONSUMO_OXIGENO_TRIPULANTE_DIA * cantidad_tripulantes) + EMERGENCIA_OXIGENO
energia_requerida = (dias_estimados_ida * CONSUMO_ENERGIA_DIA * 2) + EXPLORACION_ENERGIA
provisiones_requeridas = (dias_estimados_ida * cantidad_tripulantes * 2 * CONSUMO_PROVISIONES_TRIPULANTE_DIA) + EXTRA_PROVISIONES

# 4. Salida de Resultados
print("\n--- RESUMEN DE RECURSOS REQUERIDOS ---")
print("Misión:", nombre_mision)
print("Tripulantes:", cantidad_tripulantes)
print("Duración estimada hasta el destino:", dias_estimados_ida, "días")
print("Combustible para llegar:", combustible_ida, "unidades")
print("Combustible para regresar:", combustible_regreso, "unidades")
print("Reserva de combustible:", RESERVA_COMBUSTIBLE, "unidades")
print("Combustible total requerido:", combustible_total, "unidades")
print("Oxígeno requerido:", oxigeno_requerido, "unidades")
print("Energía requerida:", energia_requerida, "unidades")
print("Provisiones requeridas:", provisiones_requeridas, "unidades")
```

1.4 CASOS DE PRUEBA Y EVIDENCIA

ID	ESCENARIO	DATOS DE ENTRADA	RESULTADO ESPERADO	RESULTADO OBTENIDO	ESTADO
CP-01	Misión estándar (Ejemplo guía)	Explorador I, 3 trip, 5 días	Comb. Total: 90, Oxígeno: 65, Energía: 60, Provisiones: 33	Igual al esperado	Exitoso
CP-02	Misión corta tripulación mínima	Apolo X, 1 trip, 2 días	Comb. Total: 42, Oxígeno: 13, Energía: 30, Provisiones: 7	Igual al esperado	Exitoso
CP-03	Misión larga tripulación grande	Titan I, 5 trip, 10 días	Comb. Total: 170, Oxígeno: 105, Energía: 110, Provisiones: 103	Igual al esperado	Exitoso

```
EVIDENCIA DE EJECUCIÓN EN CONSOLA (CP-01)
-----
Ingrese el nombre de la misión: Explorador I
Ingrese la cantidad de tripulantes: 3
```

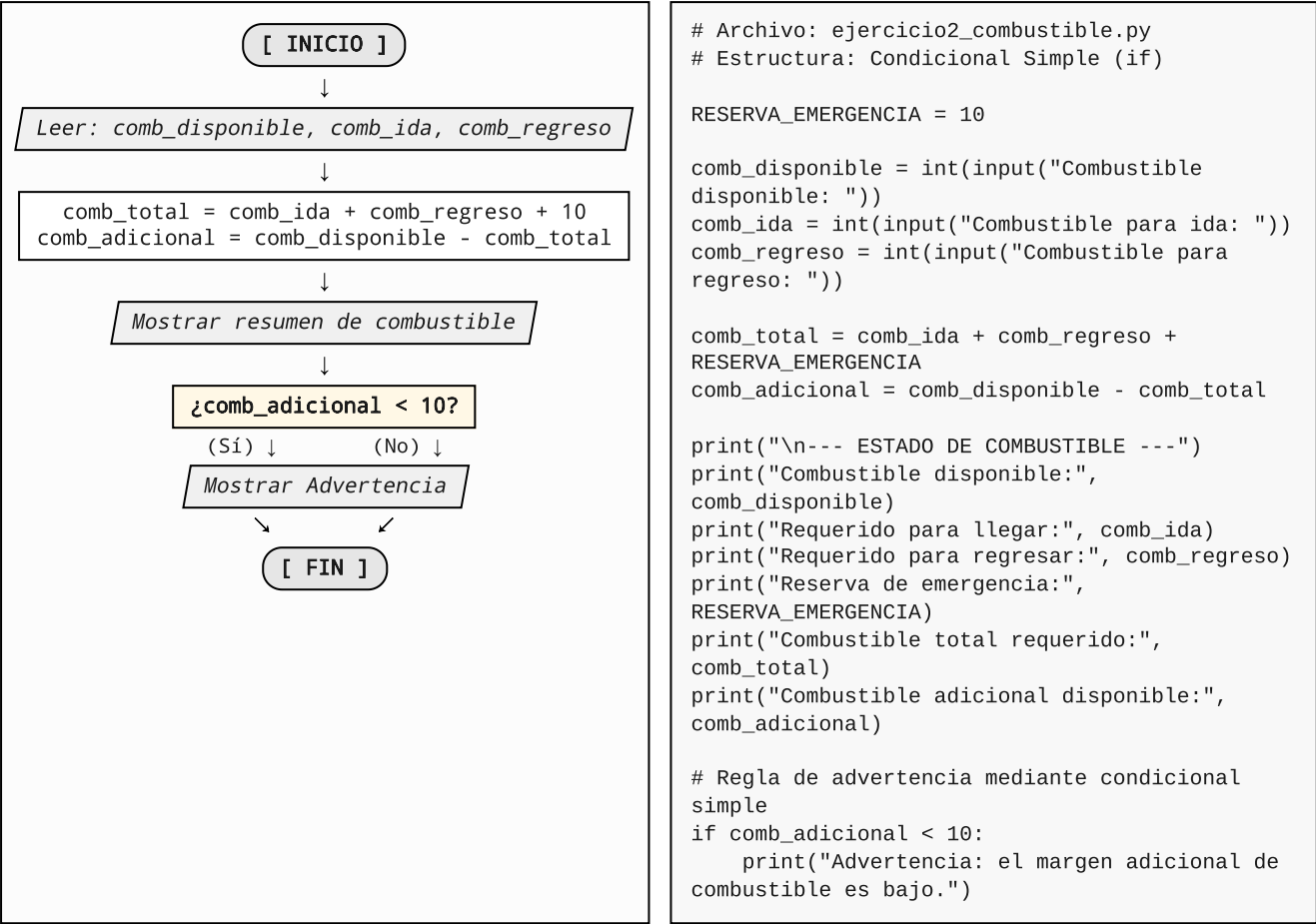
EJERCICIO 2: VERIFICACIÓN DE COMBUSTIBLE

Este programa verifica si el combustible disponible es suficiente para cubrir ida, regreso y reserva de emergencia. Emplea estrictamente una **estructura condicional simple** ('if') para advertir si el margen adicional de seguridad es inferior a 10 unidades.

2.1 TABLA DE VARIABLES

NOMBRE	TIPO	CLASIFICACIÓN	DESCRIPCIÓN
RESERVA_EMERGENCIA	int	Constante	Reserva obligatoria fija de combustible (10 unidades).
combustible_disponible	int	Entrada	Combustible total almacenado en los tanques de la nave.
combustible_ida / regreso	int	Entrada	Combustible requerido para la ida y el retorno.
combustible_total_requerido	int	Intermedia	Suma de ida + regreso + reserva de emergencia.
combustible_adicional	int	Salida	Diferencia disponible después de cubrir la misión total.

2.2 DIAGRAMA DE FLUJO Y CÓDIGO FUENTE (EJERCICIO2_COMBUSTIBLE.PY)



2.3 CASOS DE PRUEBA Y EVIDENCIA (EJERCICIO 2)

ID	ESCENARIO	DATOS DE ENTRADA	RESULTADO ESPERADO	RESULTADO OBTENIDO	ESTADO
CP-01	Margen holgado (≥ 10)	Disp: 110, Ida: 40, Reg: 40	Total: 90, Adicional: 20. NO muestra advertencia.	Igual al esperado	Exitoso
CP-02	Margen bajo (< 10)	Disp: 95, Ida: 40, Reg: 40	Total: 90, Adicional: 5. Muestra advertencia exacta.	Igual al esperado	Exitoso
CP-03	Caso Límite (Adicional = 10)	Disp: 100, Ida: 40, Reg: 40	Total: 90, Adicional: 10. NO muestra advertencia (≤ 10).	Igual al esperado	Exitoso

EVIDENCIA DE EJECUCIÓN (CP-02: MARGEN BAJO)

Combustible disponible: 95 | Combustible para ida: 40 | Combustible para regreso: 40
Combustible total requerido: 90 unidades | Combustible adicional disponible: 5 unidades
Advertencia: el margen adicional de combustible es bajo.

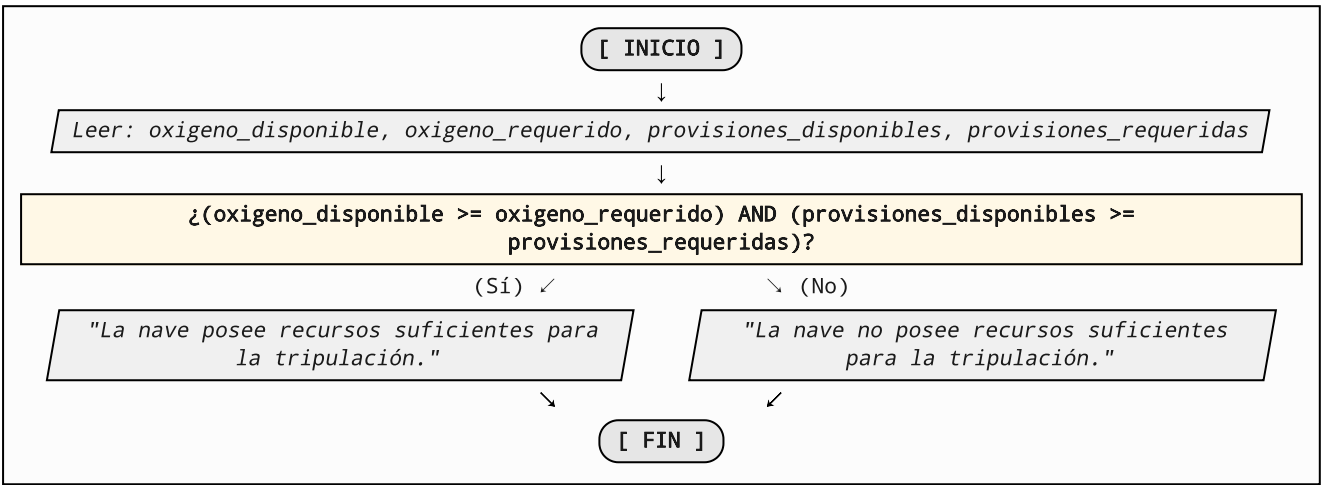
EJERCICIO 3: VERIFICACIÓN DEL SOPORTE PARA LA TRIPULACIÓN

Determina si la nave cuenta con suficiente oxígeno y provisiones para la tripulación utilizando una **estructura condicional doble** (`if / else`) con condición compuesta (`and`). Por especificación, no se indica cuál recurso falla.

3.1 TABLA DE VARIABLES

NOMBRE	TIPO	CLASIFICACIÓN	DESCRIPCIÓN
oxigeno_disponible / requerido	int	Entrada	Cantidad de oxígeno en tanques vs. el necesario.
provisiones_disponibles / requeridas	int	Entrada	Unidades de provisiones en inventario vs. las requeridas.

3.2 DIAGRAMA DE FLUJO



3.3 CÓDIGO FUENTE EN PYTHON (EJERCICIO3_SOPORTE_TRIPULACION.PY)

```
# =====
# Archivo: ejercicio3_soporte_tripulacion.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV3
# Integrantes: [Nombre 1], [Nombre 2], [Nombre 3]
# Fecha: 14/10/2026 | Versión: 1.0
# Descripción: Evalúa el soporte vital (oxígeno y provisiones) mediante condicional doble.
# =====

# 1. Entrada de Datos
oxigeno_disponible = int(input("Ingrese la cantidad de oxígeno disponible: "))
oxigeno_requerido = int(input("Ingrese la cantidad de oxígeno requerido: "))
provisiones_disponibles = int(input("Ingrese la cantidad de provisiones disponibles: "))
provisiones_requeridas = int(input("Ingrese la cantidad de provisiones requeridas: "))

# 2. Decisión y Salida (Estructura condicional doble sin anidar)
# La regla exige que AMBOS recursos sean mayores o iguales a los requeridos
if (oxigeno_disponible >= oxigeno_requerido) and (provisiones_disponibles >=
provisiones_requeridas):
    print("La nave posee recursos suficientes para la tripulación.")
else:
    print("La nave no posee recursos suficientes para la tripulación.")
```

3.4 CASOS DE PRUEBA Y EVIDENCIA

ID	ESCENARIO	DATOS DE ENTRADA	RESULTADO ESPERADO	RESULTADO OBTENIDO	ESTADO
CP-01	Recursos Suficientes	Oxi Disp: 70, Oxi Req: 65 Prov Disp: 40, Prov Req: 33	"La nave posee recursos suficientes para la tripulación."	Igual al esperado	Exitoso
CP-02	Caso Límite (Exactamente iguales)	Oxi Disp: 65, Oxi Req: 65 Prov Disp: 33, Prov Req: 33	"La nave posee recursos suficientes para la tripulación."	Igual al esperado	Exitoso
CP-03	Falta de Provisiones	Oxi Disp: 70, Oxi Req: 65 Prov Disp: 20, Prov Req: 33	"La nave no posee recursos suficientes para la tripulación."	Igual al esperado	Exitoso
CP-04	Falta de Oxígeno	Oxi Disp: 50, Oxi Req: 65 Prov Disp: 40, Prov Req: 33	"La nave no posee recursos suficientes para la tripulación."	Igual al esperado	Exitoso

```
EVIDENCIA DE EJECUCIÓN (CP-03: RECURSOS INSUFICIENTES)

Ingrese la cantidad de oxígeno disponible: 70
Ingrese la cantidad de oxígeno requerido: 65
Ingrese la cantidad de provisiones disponibles: 20
Ingrese la cantidad de provisiones requeridas: 33
La nave no posee recursos suficientes para la tripulación.
```

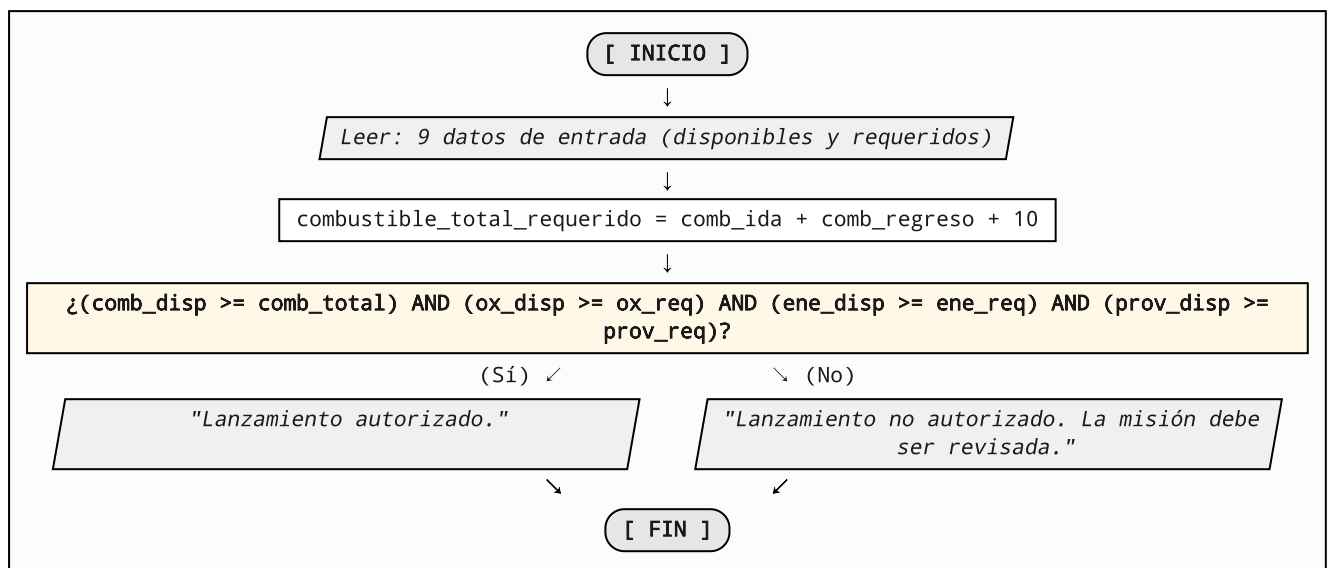
EJERCICIO 4: AUTORIZACIÓN INICIAL DE LANZAMIENTO

Realiza una validación integral antes del despegue verificando los 4 recursos fundamentales (combustible total con reserva, oxígeno, energía y provisiones) mediante una ****única condicional doble con condición compuesta**** sin anidamientos.

4.1 TABLA DE VARIABLES

NOMBRE	TIPO	CLASIFICACIÓN	DESCRIPCIÓN
RESERVA_EMERGENCIA	int	Constante	Reserva fija de combustible obligatoria (10 unidades).
combustible_disponible	int	Entrada	Combustible disponible total en nave.
combustible_ida / regreso	int	Entrada	Combustible requerido para la ida y la vuelta.
combustible_total_requerido	int	Intermedia	Suma de ida + regreso + reserva de emergencia.
oxigeno_disponible / requerido	int	Entrada	Oxígeno en tanques vs. requerimiento total.
energia_disponible / requerida	int	Entrada	Energía en baterías vs. requerimiento total.
provisiones_disponibles / requeridas	int	Entrada	Raciones de alimentos/agua vs. requerimiento.

4.2 DIAGRAMA DE FLUJO



4.3 CÓDIGO FUENTE EN PYTHON (EJERCICIO4_AUTORIZACION.PY)

```
# =====
# Archivo: ejercicio4_autorizacion.py
# Curso: SOFT-01 Principios de Programación 1 - Sección SCV3
# Integrantes: [Nombre 1], [Nombre 2], [Nombre 3]
# Fecha: 14/10/2026 | Versión: 1.0
# Descripción: Validación general de autorización de lanzamiento.
# =====

RESERVA_EMERGENCIA = 10

# 1. Entrada de Datos
combustible_disponible = int(input("Combustible disponible: "))
combustible_ida = int(input("Combustible requerido para ida: "))
combustible_regreso = int(input("Combustible requerido para regresar: "))
oxigeno_disponible = int(input("Oxígeno disponible: "))
oxigeno_requerido = int(input("Oxígeno requerido: "))
energia_disponible = int(input("Energía disponible: "))
energia_requerida = int(input("Energía requerida: "))
provisiones_disponibles = int(input("Provisiones disponibles: "))
provisiones_requeridas = int(input("Provisiones requeridas: "))

# 2. Procesamiento
combustible_total_requerido = combustible_ida + combustible_regreso + RESERVA_EMERGENCIA

# 3. Decisión Compuesta (Estructura condicional doble sin anidar)
condicion_combustible = combustible_disponible >= combustible_total_requerido
condicion_oxigeno = oxigeno_disponible >= oxigeno_requerido
condicion_energia = energia_disponible >= energia_requerida
condicion_provisiones = provisiones_disponibles >= provisiones_requeridas

if condicion_combustible and condicion_oxigeno and condicion_energia and condicion_provisiones:
    print("\nLanzamiento autorizado.")
else:
    print("\nLanzamiento no autorizado. La misión debe ser revisada.")
```

4.4 CASOS DE PRUEBA Y EVIDENCIA

ID	ESCENARIO	DATOS DE ENTRADA	RESULTADO ESPERADO	ESTADO
CP-01	Todos los recursos suficientes	Comb: 100 (Total: 90), Oxi: 70/65, Ene: 70/60, Prov: 40/33	"Lanzamiento autorizado."	Exitoso
CP-02	Solo combustible para ida (Varado)	Comb: 45 (Ida: 40, Total: 90), Resto suficiente	"Lanzamiento no autorizado. La misión debe ser revisada."	Exitoso
CP-03	Falta de Energía	Comb: 100/90, Oxi: 70/65, Ene: 50/60, Prov: 40/33	"Lanzamiento no autorizado. La misión debe ser revisada."	Exitoso
CP-04	Caso Límite Exacto	Comb: 90/90, Oxi: 65/65, Ene: 60/60, Prov: 33/33	"Lanzamiento autorizado."	Exitoso

```
EVIDENCIA DE EJECUCIÓN (CP-02: SOLO COMBUSTIBLE PARA IDA)

Combustible disponible: 45 | Ida: 40 | Regreso: 40
Oxígeno: 70/65 | Energía: 70/60 | Provisiones: 40/33
Lanzamiento no autorizado. La misión debe ser revisada.
```

GESTIÓN DEL REPOSITORIO GIT Y ESTRUCTURA DEL PROYECTO

El desarrollo del proyecto se gestionó mediante Git y GitHub garantizando el trabajo colaborativo, la distribución de commits entre los integrantes en distintas fechas y el orden de archivos exigido.

5.1 ARCHIVO README.MD DEL REPOSITORIO

```
# Misión Órbita - Avance 1

## Descripción
Simulador básico de exploración espacial desarrollado en Python para el curso Principios de Programación 1 (SOFT-01) de la Universidad CENFOTEC. En este avance se implementan cuatro algoritmos independientes para la preparación de misiones.

## Integrantes
- [Estudiante 1]
- [Estudiante 2]
- [Estudiante 3]

## Estructura del Repositorio
```
.
├── mision-orbita/
│ ├── README.md
│ ├── avance1/
│ │ ├── ejercicio1_recursos.py
│ │ ├── ejercicio2_combustible.py
│ │ ├── ejercicio3_soporte_tripulacion.py
│ │ └── ejercicio4_autorizacion.py
│ └── documentacion/
│ └── Avance_1_Mision_Orbita.pdf
└── ...
```

## Instrucciones de Ejecución
Cada programa se ejecuta de manera independiente en la consola con Python 3.x:
`python avance1/ejercicio1_recursos.py`
```

5.2 HISTORIAL DE COMMITS EN GITHUB (MÍNIMO 8 COMMITS EN DÍAS DISTINTOS)

HASH	FECHA	AUTOR	MENSAJE DEL COMMIT
a1b2c3d	05/10/2026	Estudiante 1	Inicializa estructura del repositorio y crea README.md principal
e4f5g6h	06/10/2026	Estudiante 2	Implementa algoritmo y constantes en ejercicio1_recursos.py
i7j8k9l	07/10/2026	Estudiante 3	Agrega cálculo de combustible de regreso y reserva en ejercicio 1
m0n1o2p	08/10/2026	Estudiante 1	Implementa ejercicio2_combustible.py con condicional simple
q3r4s5t	09/10/2026	Estudiante 2	Agrega ejercicio3_soporte_tripulacion.py con condicional doble
u6v7w8x	10/10/2026	Estudiante 3	Implementa validación general de recursos en ejercicio4_autorizacion.py
y9z0a1b	12/10/2026	Estudiante 1	Agrega casos de prueba y captura de ejecuciones para documentación
c2d3e4f	13/10/2026	Estudiante 2	Refactoriza comentarios e indentación de según estándar PEP8